DevOps-Based Deployment and Orchestration

of a Multi-Agent Insurance System

Course Name: DevOps

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Mustafa shaikh	EN22CS301624
02	Naman Mehta	EN22CS301629
03	Naman Raghuvanshi	EN22CS301632
04	Nandini Sharma	EN22CS301635
05	Nishkarsh Sharma	EN22CS301659

Group Name: DO-20

Project Number: 09D6

Industry Mentor Name: Vaibhav Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

1. Introduction

This document presents the **Low-Level Design (LLD)** of the Multi-Agent Insurance System deployment using DevOps practices. It describes the detailed technical components involved in containerizing and deploying the application. The design focuses on how Docker is used to package the application and how Kubernetes manages the deployment, scaling, and updating of containers within the cluster environment.

1.1 Scope of the Document

The scope of this document is to describe the **detailed design and configuration** of the system components used in the deployment process. It includes the structure of Docker images, Kubernetes deployment configurations, and service definitions used to run the application.

This document mainly focuses on the **technical implementation details of the deployment architecture**, which helps developers and DevOps engineers understand how the system is configured and managed in the Kubernetes environment.

1.2 Intended Audience

- **Developers** – To understand the detailed system deployment and configuration.
- **DevOps Engineers** – To manage containerization, deployment, and orchestration using Docker and Kubernetes.
- **System Administrators** – To maintain and monitor the deployed application environment.
- **Project Evaluators / Reviewers** – To review the technical design and implementation of the system.
- **Students / Team Members** – To understand the deployment workflow and system structure.

1.3 System Overview

The Multi-Agent Insurance System is designed to handle insurance-related queries through a collaborative agent-based architecture. The system consists of multiple agents that work together to process user requests and generate appropriate responses. Each agent is responsible for a specific task within the workflow.

The system processes user queries, retrieves relevant information, and generates structured outputs through coordinated interaction between the agents. The architecture follows a

modular design that allows different components of the system to operate independently while collaborating to complete the overall task.

This design improves system organization, maintainability, and the ability to extend the system with additional functionalities in the future.

2. System Design

The system design describes how the application is deployed and managed using DevOps practices. The system is containerized and deployed in a controlled environment to ensure consistent execution across different platforms. The design includes container creation, deployment configuration, and service management to run the application efficiently.

The system components are organized in a way that supports automated deployment, scalability, and easier management of application services.

2.1 Application Design

The application design focuses on how the system is packaged and deployed within the infrastructure environment.

Main components include:

- **Source Code Repository** – Stores the application code and configuration files.
- **Container Configuration** – Defines how the application is packaged into a container image.
- **Container Image** – A packaged version of the application with all required dependencies.
- **Deployment Configuration** – Defines how the container runs in the cluster environment.
- **Service Layer** – Exposes the application so users can access it.

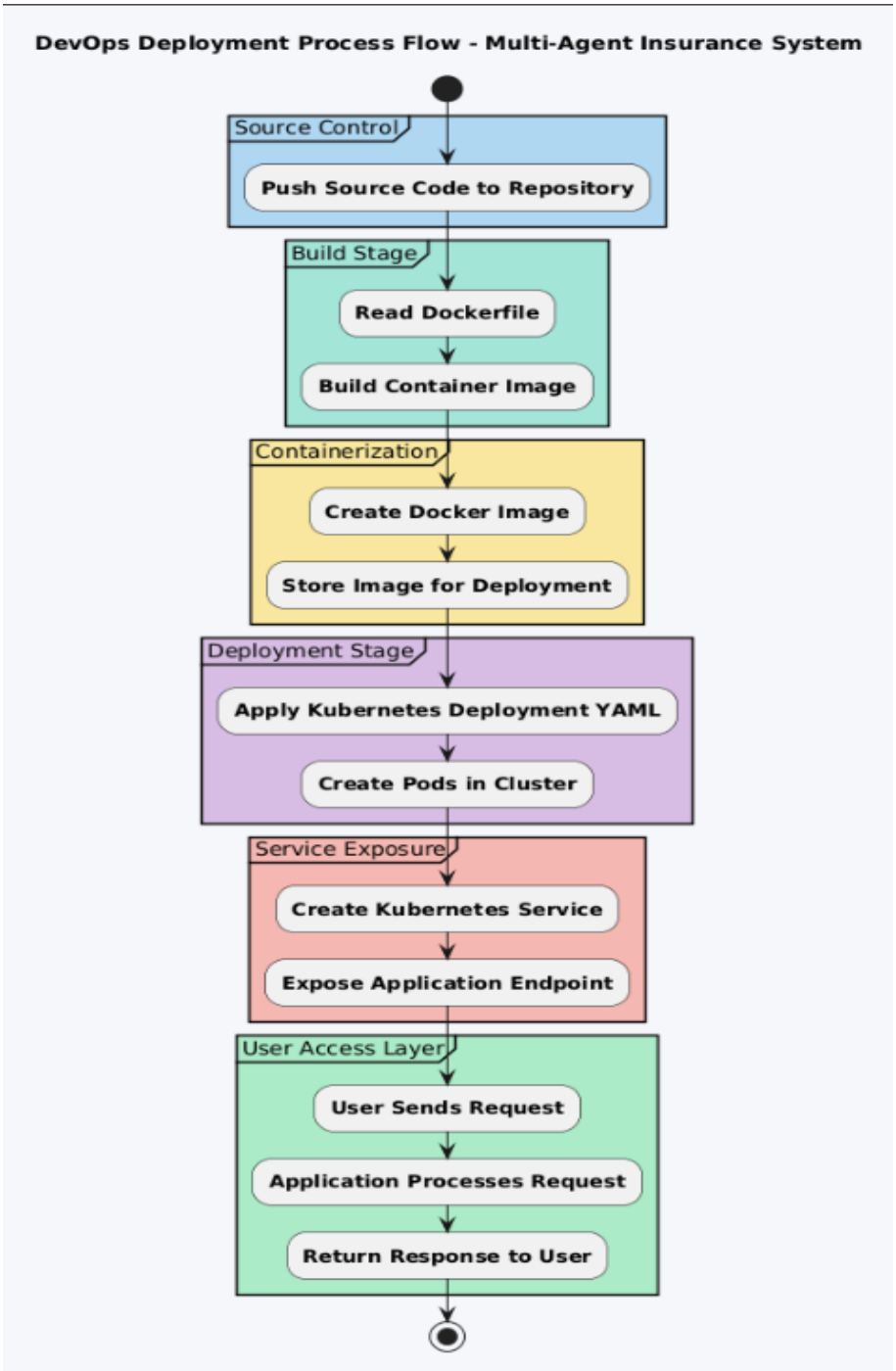
This design ensures that the application can be deployed consistently, managed efficiently, and updated without affecting system availability.

1.2. Process Flow

The process flow describes the sequence of steps involved in building and deploying the application within the system environment.

Process Flow Steps:

- **Code Management** – The application source code is stored and maintained in the repository.
- **Build Process** – The application is packaged into a container image using the configuration file.
- **Image Creation** – The container image is created with all required dependencies.
- **Deployment** – The container image is deployed into the cluster environment using deployment configuration files.
- **Service Exposure** – The application service is exposed so that it can be accessed by users.
- **Application Access** – Users interact with the deployed application through the provided service endpoint.



1.3 Information Flow

Information flows from the **source code repository (Git)** through the **automation and deployment pipeline**, where container images are built and deployed to the target infrastructure. The deployment configurations define how the application runs inside the container environment.

The deployment system manages the container lifecycle and ensures that the application is properly created, updated, and maintained in the runtime environment.

1.4 Components Design

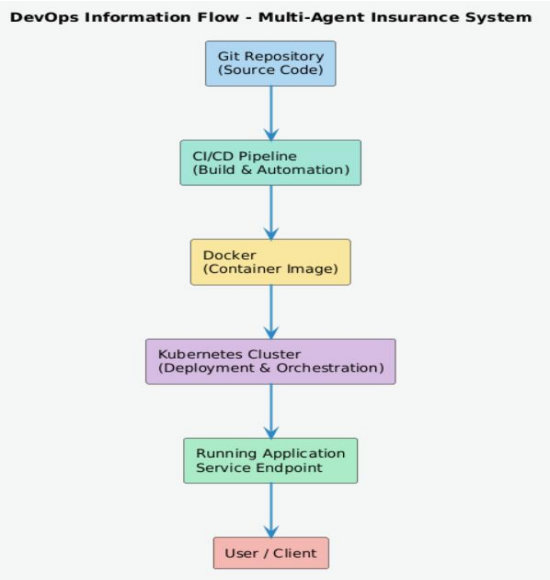
• **Git Repository** – Stores the application source code and configuration files.
• **Automation / CI Pipeline** – Triggers build and deployment processes when code changes occur.
• **Docker** – Packages the application and its dependencies into container images.
• **Kubernetes** – Manages container deployment, scaling, and service exposure.

1.5 Key Design Considerations

• **Modularity** – System components are organized into separate configurations for easier maintenance.
• **Scalability** – The container orchestration platform allows the application to scale according to workload.
• **Reliability** – Deployment configurations ensure consistent application execution across environments.

1.6 API Catalogue

• **Kubernetes API** – Used to manage deployments, services, and container lifecycle operations within the cluster.



2. Data Design

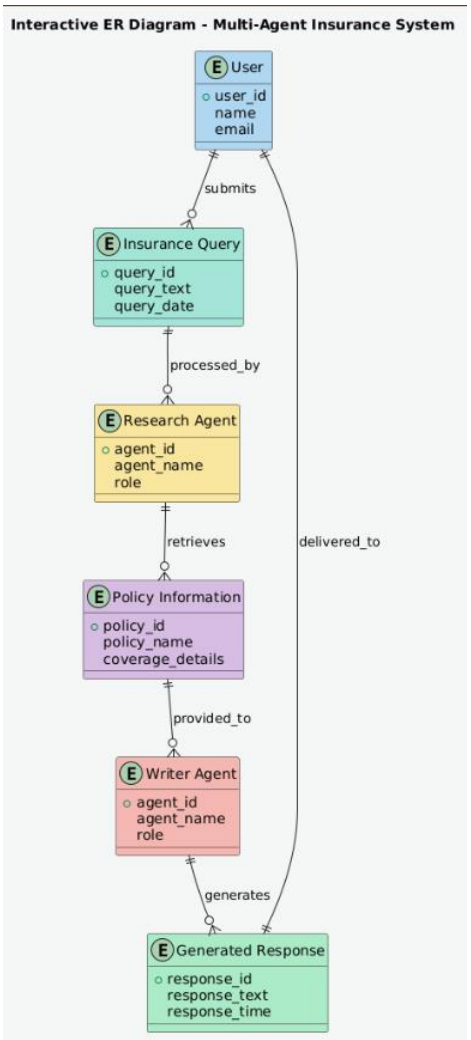
Data design describes how the system manages and organizes the data used during application execution and deployment. It defines how configuration files, container images, and deployment resources are structured and stored so that the system can operate efficiently.

2.1 Data Model

The data model defines the structure of the information used within the system components.

Main data elements include:

• **Source Code Files** – Contains the application logic and configuration files stored in the repository.
• **Container Configuration** – Defines how the application is packaged into a container image.
• **Container Image** – A packaged version of the application along with its dependencies.
• **Deployment Configuration Files** – YAML files that define how containers are deployed and managed.
• **Service Configuration** – Defines how the application is exposed for external access.



The system accesses information through structured interactions between agents and internal data sources. When a user submits a query related to insurance policies, the system processes the request through different agents.

The **Research Agent** retrieves relevant insurance policy information from the available knowledge source. After retrieving the required data, it forwards the information to the **Writer Agent**, which processes the retrieved content and generates a structured response. The final response is then delivered back to the user.

This mechanism ensures that data retrieval and response generation are handled efficiently through collaboration between specialized agents.

2.3 Data Retention Policies

• **Query History** – User queries may be stored temporarily during system processing to generate responses.
• **Generated Responses** – The system may maintain generated responses for reference and system evaluation.
• **Agent Processing Logs** – Basic logs of agent interactions can be maintained to monitor system performance and debugging.

These policies help maintain system reliability and allow developers to analyze system behavior when required.

2.4 Data Migration

Data migration in the system mainly involves updating or modifying the knowledge sources used by the agents. When new insurance policy information becomes available, the knowledge base can be updated so that the Research Agent can retrieve the latest information.

The modular structure of the system allows new data sources or updated policy information to be integrated without affecting the overall system workflow.

3. Interfaces

- **User Interface**: Jenkins Web Dashboard for monitoring pipeline status.
- **Command Line Interface (CLI)**: Terraform and Git CLI for developers.

4. State and Session Management

State management is handled through the **Terraform State File** (.tfstate). This file acts as a database that records the current state of the infrastructure, ensuring that Terraform knows which resources to create, update, or delete during a run.

5. Caching

- **Docker Layers**: Jenkins uses Docker layer caching to speed up the building of telecom service images.
- **Terraform Plugins**: Provider plugins are cached locally to reduce deployment time.

6. Functional Requirements

The functional requirements describe the DevOps capabilities used to build, deploy, and manage the Multi-Agent Insurance Recommendation System.

6.1 Source Code Management

• **Version Control** – The project source code is maintained in a Git repository to track changes and manage collaboration.
• **Branch Management** – Developers can create branches to develop features and merge them into the main branch after testing.
• **Code Tracking** – Git maintains a history of changes for debugging and development purposes.

6.2 Application Containerization

• **Dockerfile Creation** – The application is packaged using a Dockerfile to create a portable container image.
• **Environment Consistency** – The Docker container ensures that the application runs consistently across development and deployment environments.
• **Dependency Management** – All required Python dependencies from the `requirements.txt` file are installed within the container.

6.3 Continuous Integration (CI)

• **Automated Build Process** – Each code update triggers an automated build process.
• **Dependency Installation** – The CI pipeline installs required dependencies before running the application.
• **Application Testing** – Basic checks ensure that the Flask application runs successfully.

6.4 Continuous Deployment (CD)

• **Automated Deployment** – After successful builds, the application container can be deployed automatically.
• **Container Execution** – The system runs the containerized Flask application on the target environment.

• **Service Availability** – The deployed application becomes accessible through a web interface.

6.5 Monitoring and Management

• **Application Logs** – System logs help monitor application activity and detect errors.
• **Deployment Monitoring** – Developers can track the status of builds and deployments through pipeline tools.
• **System Updates** – Updated application versions can be redeployed without manual configuration.

7. Non-Functional Requirements

These requirements describe the performance, reliability, and maintainability aspects of the DevOps pipeline.

7.1 Security Aspects

• **Secure Dependency Installation** – Dependencies are installed using verified package sources.
• **Access Control** – Only authorized users can modify the source code repository.
• **Configuration Security** – Sensitive environment variables can be managed securely during deployment.

7.2 Performance Aspects

• **Fast Build and Deployment** – Automation reduces the time required to deploy application updates.
• **Efficient Resource Usage** – Containerized applications use system resources efficiently.
• **Scalable Deployment** – The system can support scaling by deploying multiple container instances.

7.3 Reliability and Maintainability

• **Reproducible Environment** – Docker containers ensure consistent environments across development and production.
• **Automated Workflow** – CI/CD pipelines automate the build and deployment process.
• **Easy Maintenance** – Updates and fixes can be deployed quickly using automated pipelines.

8. References

- Problem Statement: **Deploy Insurance App to Kubernetes**.

  Project Group: **D6 - Group 09**.

Github Repo Link : https://github.com/Nandinisharma-04Git/Multi-Agent_Insurance_System